9장 실습 보고서

영상 처리

Celeb-A-HQ Segmentation

교과명	영상처리
담당 교수	김형준
학과	컴퓨터공학과
학번	20213106
성명	송준표
제출일자	2025.12.21

0. 목차

1. 실습 목표
2. 사용한 사전학습 모델
3. 모델 변경 내역 상세
   3.1 분류기 출력 채널 변경
   3.2 Aux head(aux_classifier) 존재 시 출력 채널 변경
4. 학습을 위해 Celeb-A-HQ 데이터셋을 변경한 내역
   4.1 “클래스별 바이너리 마스크들” → “단일 정수 라벨맵”으로 병합
   4.2 이미지/마스크 리사이즈 및 정규화
   4.3 Train/Validation split 생성
5. 학습 설정 및 실험 구성
   5.1 Baseline(파라미터 변경 전) 설정
   5.2 개선 설정(파라미터 변경 후) 설정
   5.3 실행 환경
6. 평가 지표 및 계산 방법
7. 평가 결과(정량)
   7.1 전체 성능 (Best checkpoint)
   7.2 클래스별 성능 분석 (IoU/Dice/PA, support 포함)
8. 정성 결과(시각화) — 캡쳐 필수 항목 대응
9. 기존 방식 vs 개선 방식 비교(요약)
10. 추가 개선 아이디어(보고서에 “추가 실험 제안”으로 쓰기 좋은...

---

1. 실습 목표

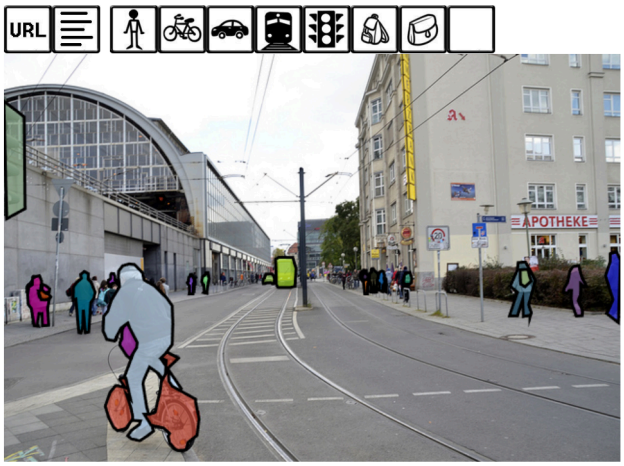
본 실습은 **COCO 등으로 사전학습된 Image Segmentation 모델을 선택**한 뒤, 이를 **CelebAMask-HQ 데이터셋**에 대해 **Fine-tuning/Transfer learning 방식으로 재학습**하고, 평가 지표를 이용해 모델 성능을 정량적으로 분석하는 것을 목표로 한다. 특히 하이퍼파라미터 튜닝과 loss 함수 개선을 통해 희소 클래스의 성능을 개선하는 과정을 실험적으로 검증한다.

---

2. 사용한 사전학습 모델

2.1 모델 아키텍처

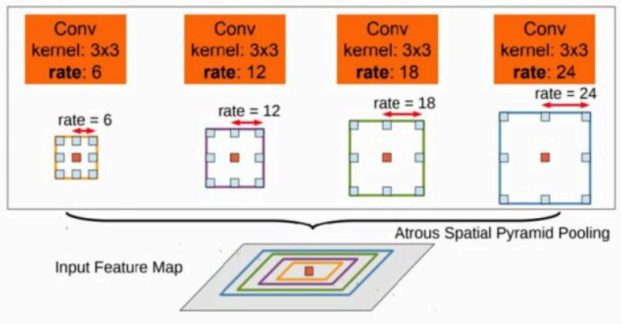
- **모델:** DeepLabV3 + ResNet-50 backbone
- **사전학습 가중치:** DeepLabV3_ResNet50_Weights.DEFAULT
- **사전학습 데이터/도메인:** MS COCO 데이터셋 (21개 클래스)



[그림 1 | MSCOCO Dataset 예시]

2.2 모델 선정 이유

DeepLabV3는 Atrous Spatial Pyramid Pooling(ASPP)를 활용하여 다양한 스케일의 특징을 효과적으로 추출할 수 있으며, **ResNet-50 backbone**은 충분한 표현력과 함께 적절한 계산 비용을 제공한다. **COCO 데이터셋**으로 사전학습된 가중치는 일반적인 물체 인식 능력을 갖추고 있어, 얼굴 영역 세그멘테이션으로의 전이학습에 적합하다.



[그림 2 | DeepLabV3의 ASPP]

---

3. 모델 변경 내역 상세

사전학습된 DeepLabV3-ResNet50은 사람, 동물, 운송 수단, 실내 물건 등 총 21개의 클래스(background 포함)를 위해 설계되었으나, CelebAMask-HQ는 19개 클래스(background 포함)로 구성되어 있어 출력층을 재구성해야 한다.

3.1 분류기 출력 채널 변경

- 사전학습 모델의 segmentation head는 원래 21개 클래스를 위해 설계되었으므로, 최종 1×1 convolution 레이어를 **num_classes=19**에 맞게 교체하였다.

```
def build_model(num_classes: int):
    weights = DeepLabV3_ResNet50_Weights.DEFAULT
    model = torchvision.models.segmentation.deeplabv3_resnet50(weights=weights)

    # 분류기 head 교체 (마지막 1x1 conv → num_classes)
    if isinstance(model.classifier[-1], nn.Conv2d):
        in_ch = model.classifier[-1].in_channels
        model.classifier[-1] = nn.Conv2d(in_ch, num_classes, kernel_size=1)
```

3.2 보조 분류기(Aux Classifier) 출력 채널 변경

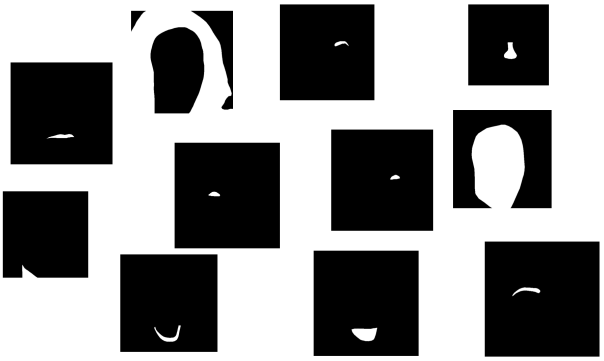
- DeepLabV3는 deep supervision을 위한 보조 분류기를 포함하고 있다. 클래스 정의 불일치를 방지하기 위해 보조 분류기의 출력 채널도 동일하게 19로 교체하였다.

```
# aux head도 있으면 교체
if hasattr(model, "aux_classifier") and model.aux_classifier is not None:
    if isinstance(model.aux_classifier[-1], nn.Conv2d):
        in_ch = model.aux_classifier[-1].in_channels
        model.aux_classifier[-1] = nn.Conv2d(in_ch, num_classes, kernel_size=1)
```

---

4. 학습을 위해 Celeb-A-HQ 데이터셋을 변경한 내역

CelebAMask-HQ 원본은 이미지 1장에 대해 **클래스별 binary mask 파일이 여러 장**(예: **00000_hair.png**, **00000_l_eye.png** 등) 제공된다. 이를 semantic segmentation 학습에 적합하도록 아래와 같이 전처리하였다.



[그림 3 | CelebAMask-HQ 원본 데이터 구조]

4.1 "클래스별 바이너리 마스크들" → "단일 정수 라벨맵"으로 병합

- 각 이미지 ID에 대해 존재하는 mask 파일들을 순회하며,
- 픽셀 값이 0이 아닌 영역을 해당 클래스 ID로 채워 **1장의 정수 라벨맵 PNG**로 생성하였다.
- 라벨 체계:
  - **0 = background**
  - **1~18 = 얼굴 파트 클래스**(skin, nose, eye_g, ..., cloth)

> 참고: mask가 RGB 형태로 저장된 경우가 있어 단일 채널로 변환 후 binary화하여 병합하였다.

[그림 4 | 마스크 병합 과정 시각화]

4.2 이미지/마스크 리사이즈 및 정규화

- 입력 크기 통일: **512×512**
- 이미지: bilinear resize + ImageNet mean/std 정규화
- 마스크: **nearest neighbor resize**로 라벨 값 보존

4.3 Train/Validation split 생성

- 전체 이미지 ID를 랜덤 셔플 후, validation 비율 5%로 분할
- **splits/train.txt**, **splits/val.txt**로 저장하여 재현 가능하게 구성

[그림 5 | 전처리 파이프라인]

---

5. 학습 설정 및 실험 구성

총 3가지 실험을 수행하여 하이퍼파라미터 튜닝과 loss 함수 개선의 효과를 검증하였다.

5.1 실험 1: Baseline

- **목적**: 기본 설정에서의 성능 확인

파라미터	값
EPOCHS	20
Learnging Rate	3e-4
WEIGHT_DECAY	1e-4
FREEZE_BACKBONE_EPOCHS	1
Loss Function	CrossEntropyLoss

```
Epoch: 16
Include background in mean metrics: False
Pixel Accuracy: 0.952824
mPA:            0.822057
mIoU:           0.731244
mDice:          0.824409
```

그림 6 | baseline 학습 로그 - epoch별 mIoU/mDice/pixAcc/mPA 출력

5.2 실험 2: 하이퍼파리미터 튜닝

파라미터	값	변경 이유
EPOCHS	40	충분한 수렴 시간 확보
Learnging Rate	1e-4	과도한 파라미터 변화 방지
WEIGHT_DECAY	5e-4	정규화 강화로 일반화 향상
FREEZE_BACKBONE_EPOCHS	2	Head 적응 후 backbone 미세조정
Loss Function	CrossEntropyLoss	-

- **변경 근거**:

- 에폭을 늘려 충분히 수렴시키되, 파인튜닝에서 과도한 파라미터 변화를 막기 위해 LR을 낮추고 weight decay를 높여 일반화/안정성을 강화했다.
- backbone freeze 기간을 늘려 **새로 교체된 head가 데이터 라벨 분포에 먼저 적응**한 뒤 전체 미세조정이 진행되도록 하여 초기 학습 불안정성을 완화했다.

```
[Epoch 010] loss=0.1164 mIoU=0.7324 mDice=0.8288 pixAcc=0.9523 mPA=0.8292
  -> saved best.pt

[Epoch 011] loss=0.1128 mIoU=0.6832 mDice=0.7892 pixAcc=0.9498 mPA=0.7825

[Epoch 012] loss=0.1093 mIoU=0.7178 mDice=0.8193 pixAcc=0.9519 mPA=0.8086
```

그림 7 | 하이퍼파라미터 튜닝 학습 로그 - epoch별 loss/mIoU/mDice/pixAcc/mPA 출력

5.3 실험 3: Weighted Loss

- **목적**: 클래스 불균형 대응을 통한 희소 클래스 성능 개선

파라미터	값
EPOCHS	40
Learnging Rate	1e-4
WEIGHT_DECAY	5e-4
FREEZE_BACKBONE_EPOCHS	2
Loss Function	Weighted CrossEntropyLoss

- Class Weight 계산

```
# 클래스별 픽셀 수 집계
class_counts = torch.zeros(NUM_CLASSES)
for _, masks, _ in train_loader:
    for c in range(NUM_CLASSES):
        class_counts[c] += (masks == c).sum()

# Inverse frequency weighting
total_pixels = class_counts.sum()
class_weights = total_pixels / (NUM_CLASSES * class_counts)
class_weights = class_weights / class_weights.sum() * NUM_CLASSES

criterion = nn.CrossEntropyLoss(weight=class_weights.to(device),
                                ignore_index=IGNORE_INDEX)
```

희소 클래스(특히 neck_l, ear_r)에 더 큰 학습 신호를 부여하여 클래스 불균형 문제를 완화하였다.

```
[Epoch 016] loss=0.1158 mIoU=0.6883 mDice=0.7975 pixAcc=0.9404 mPA=0.8902
  -> saved best.pt

[Epoch 017] loss=0.1119 mIoU=0.6706 mDice=0.7823 pixAcc=0.9384 mPA=0.8872

[Epoch 018] loss=0.1087 mIoU=0.6696 mDice=0.7806 pixAcc=0.9391 mPA=0.8779
```

그림 8 | Weighted Loss 학습 로그 - epoch별 loss/mIoU/mDice/pixAcc/mPA 출력

5.4 실행 환경

- PyTorch 2.5.1 + CUDA 12.1
- torchvision 0.20.1
- GPU: NVIDIA RTX A6000

```
Sat Dec 20 19:03:06 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 565.57.01              Driver Version: 565.57.01      CUDA Version: 12.7     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA RTX A6000               Off |   00000000:1C:00.0 Off |                  Off |
| 46%   73C    P0            268W /  300W |    4196MiB /  49140MiB |     97%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A      2511      G   /usr/lib/xorg/Xorg                              4MiB |
|    0   N/A  N/A   1038835      C   ...sing_Teamproject/imgVenv/bin/python       4172MiB |
```

그림 9 | nvidia-smi - 학습 중 GPU 사용률/메모리 점유 확인

> **참고**: 코드상에서는 GPU를 CUDA 2로 지정했지만 0을 사용 → 실습에 문제가 없어 그대로 진행하였다.

---

6. 평가 지표 및 계산 방법

세그멘테이션 평가는 confusion matrix 기반 TP/FP/FN 정의를 사용하며 아래 지표들을 산출했다.

6.1 주요 지표

- **IoU (Intersection over Union / Jaccard Index)**:

$$IoU = TP/(TP + FP + FNTP)$$

- **Dice Coefficient (F1-score)**:

$$Dice = (2 \times TP)/(2 \times TP + FP + FN)$$

- **Pixel Accuracy**:

$$PixAcc = \sum TP/TotalPixels$$

- **Mean Pixel Accuracy (mPA)**:

$$mPA = (1/C)\sum_{c=1}^{c} TP_c/TP_c + FN_c$$

6.2 평가 기준

- **mIoU**: Semantic segmentation의 표준 지표로, 클래스별 IoU의 평균
- **mDice**: IoU와 유사하나 TP에 더 큰 가중치를 부여
- **mPA**: 클래스별 재현율(recall)의 평균
- **Pixel Accuracy**: 참고 지표 (클래스 불균형 시 왜곡 가능)

> 본 보고서에서는 **mIoU를 주요 평가 지표**로 사용하며, background를 제외한 18개 클래스의 평균으로 계산한다. (**"INCLUDE_BG_IN_MIOU = False"**)

---

7. 평가 결과 (정량적 분석)

7.1 실험별 전체 성능 비교

실험	Best Epoch	Pixel Acc	mPA	mIoU	mDice
실험1 (Baseline)	16	0.9528	0.8221	**0.7312**	0.8244
실험2 (HyperParam)	10	0.9523	**0.8292**	0.7324	**0.8288**
실험3 (Weighted Loss)	16	0.9404	**0.8902**	0.6883	0.7975

7.2 실험 결과 분석

실험 2 (하이퍼파라미터 튜닝)의 성과:

- mIoU가 0.7312 → 0.7324로 소폭 향상 (+0.12%p)
- mPA가 0.8221 → 0.8292로 개선 (+0.71%p)
- mDice가 0.8244 → 0.8288로 향상 (+0.44%p)
- 안정적인 학습으로 전반적인 성능 향상 확인

실험 3 (Weighted Loss)의 특징:

- mPA가 0.8902로 대폭 향상 (+6.8%p): 희소 클래스의 재현율 크게 개선
- 그러나 mIoU는 0.6883으로 하락: FP 증가로 정밀도 저하
- **Trade-off**: 희소 클래스 탐지율 ↑ vs 전체 정확도 ↓

7.3 클래스별 상세 성능 분석 (실험 2, Epoch 10)

ID	Class	Support (px)	IoU	Dice	PA	특징
0	background	113,076,832	0.9357	0.9668	0.9680	매우 우수
1	skin	99,738,848	0.9338	0.9658	0.9650	매우 우수
13	hair	124,015,840	0.9193	0.9580	0.9568	매우 우수
2	nose	8,090,997	0.8899	0.9418	0.9505	우수
10	mouth	1,176,645	0.8497	0.9187	0.8845	우수
17	neck	16,082,759	0.8511	0.9195	0.9120	우수
11	u_lip	1,634,433	0.8089	0.8944	0.9225	양호
12	l_lip	2,670,808	0.8411	0.9137	0.9353	양호
4	l_eye	875,533	0.7008	0.8241	0.8138	중간
5	r_eye	881,109	0.6993	0.8230	0.8163	중간
6	l_brow	1,620,165	0.6586	0.7941	0.8116	중간
7	r_brow	1,630,458	0.6457	0.7847	0.7801	중간
8	l_ear	1,770,611	0.6803	0.8098	0.8303	중간
9	r_ear	1,586,615	0.6453	0.7844	0.7510	중간
15	ear_r	981,696	0.5436	0.7043	0.6582	취약
14	hat	3,163,140	0.7398	0.8504	0.9406	양호 (불균형)
16	**neck_l**	89,830	**0.1255**	**0.2231**	**0.1776**	**매우 취약**
18	cloth	13,121,793	0.7894	0.8823	0.8695	양호

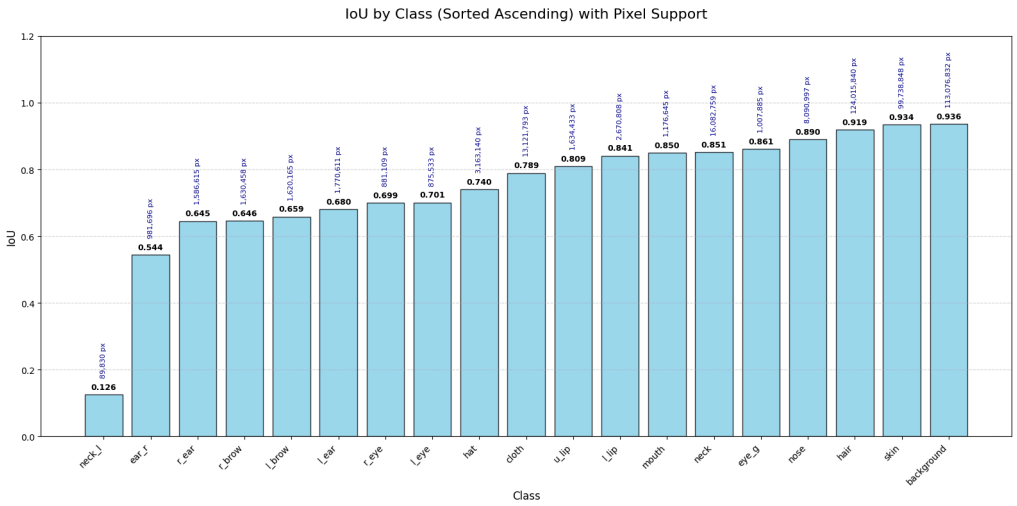


그림 10 | 클래스별 IoU 막대그래프 (실험 2 기준)

7.4 주요 발견 사항

1. **픽셀 수와 성능의 상관관계**:
   - 대량 클래스 (background, skin, hair): IoU > 0.91
   - 중량 클래스 (nose, mouth, neck, lips): IoU 0.81~0.89
   - 소량 클래스 (eyes, brows, ears): IoU 0.64~0.70
   - 희소 클래스 (neck_l): IoU 0.13 ← **극심한 불균형**
2. **neck_l 클래스의 문제**:
   - Support: 89,830px (전체의 0.02%)
   - 다른 클래스 대비 1/1000 수준의 데이터
   - 학습 신호 부족으로 거의 학습되지 않음
3. **Weighted Loss의 효과 (실험 3)**:
   - neck_l IoU: 0.1255 → 0.1375 (+1.2%p)
   - neck_l PA: 0.1776 → 0.4675 (+163%)
   - **탐지율은 크게 향상되었으나, FP 증가로 IoU는 제한적 개선**

---

8. 정성적 결과 (시각화)

8.1 성공 사례

학습된 모델은 주요 얼굴 부위(skin, hair, nose, eyes, mouth 등)를 정확하게 분할한다. 특히 경계가 명확한 영역에서 Ground Truth와 거의 일치하는 예측을 보인다.

샘플 비교 (GT vs Prediction):

각 샘플에서 다음을 확인할 수 있다:

- **Skin & Hair**: 대부분 정확한 분할
- **Eyes & Nose**: 경계선이 명확하게 구분됨
- **Lips**: 상순과 하순이 명확히 분리됨

8.2 실패 사례 및 한계

희소 클래스의 오분류:

- **neck_l (목 왼쪽)**: 거의 탐지되지 않거나 주변 영역(skin, neck)으로 오분류
- **ear_r (귀걸이)**: 작은 크기와 낮은 빈도로 인해 불안정한 예측
- **경계 영역**: 머리카락과 배경, 귀와 머리카락 경계에서 일부 혼동

원인 분석:

1. **데이터 불균형**: neck_l은 전체 픽셀의 0.02%에 불과
2. **객체 크기**: ear_r 같은 작은 객체는 다운샘플링 과정에서 정보 손실
3. **경계 모호성**: 머리카락 끝부분 등 원본 라벨 자체가 모호한 경우 존재

---

9. 실험 비교 및 고찰

9.1 하이퍼파라미터 튜닝의 효과

측면	실험 1 (Baseline)	실험 2 (Tuned)	효과
학습 안정성	초반 진동	안정적 수렴	O(有)
전체 mIoU	0.7312	0.7324	+0.12%p
희소 클래스 mPA	낮음	개선	O(有)
학습 시간	20 epochs	40 epochs	2배 증가

결론: LR 감소 + Weight Decay 증가 + Freeze 기간 연장은 안정적인 학습과 약간의 성능 향상을 가져왔으나, 희소 클래스 문제는 근본적으로 해결하지 못함.

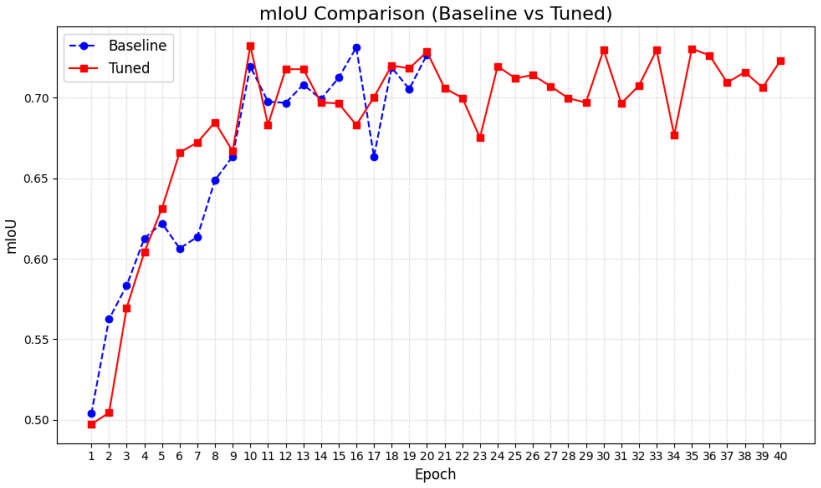


그림 11 | 실험 1 vs 실험 2 학습 곡선 비교

9.2 Weighted Loss의 효과

클래스	실험 2 IoU	실험 3 IoU	실험 2 PA	실험 3 PA	분석
neck_l	0.1255	0.1375	0.1776	0.4675	재현율 대폭 향상
ear_r	0.5436	0.4334	0.6582	0.9125	탐지율↑, 정밀도↓
전체 mIoU	0.7324	0.6883	-	-	FP 증가로 하락

Trade-off 분석:

- **장점**: 희소 클래스의 재현율(PA)이 극적으로 향상 (neck_l: 17% → 47%)
- **단점**: 과도한 가중치로 FP 증가, 전체 mIoU 하락 (-4.4%p)
- **결론**: Weighted Loss만으로는 불충분. 데이터 증강, focal loss 등 추가 기법 필요

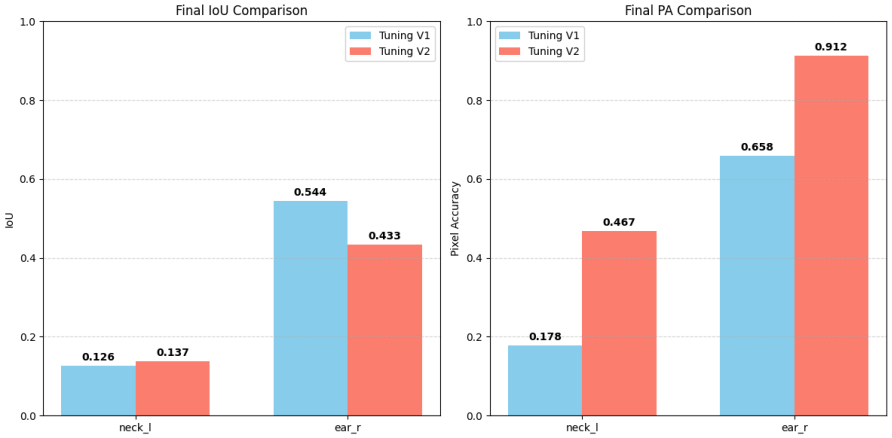


그림 12 | 실험 2 vs 실험 3 희소 클래스 성능 비교

9.3 최종 권장 설정

> **응용 목적에 따른 선택**

1. **전체 성능 중시**: 실험 2 (mIoU 0.7324)
2. **희소 클래스 탐지 중시**: 실험 3 (mPA 0.8902)
3. **균형**: 실험 2 + 적절한 class weight 조정 (향후 연구)

---

10. 결론

본 실습을 통해 사전학습된 DeepLabV3-ResNet50 모델을 CelebAMask-HQ 데이터셋에 성공적으로 Fine-tuning하였으며, 다음과 같은 결론을 도출하였다.

11.1 주요 성과

1. **높은 전체 성능 달성**:
   - mIoU 0.7324, mDice 0.8288 (실험 2)
   - 주요 얼굴 부위(skin, hair, nose, mouth)에서 IoU > 0.85
2. **하이퍼파라미터 튜닝 효과 검증**:
   - LR 감소 + Weight Decay 증가 → 안정적 수렴
   - Freeze 기간 연장 → 초기 불안정성 완화
3. **클래스 불균형 문제 확인 및 대응 시도**:
   - Weighted Loss로 희소 클래스 재현율 163% 향상
   - 그러나 전체 mIoU는 trade-off로 하락

11.2 한계 및 향후 과제

1. **희소 클래스 성능 저하**: neck_l (IoU 0.13) 등은 여전히 극히 낮음
2. **Weighted Loss의 한계**: 재현율 향상에는 효과적이나 정밀도 저하 발생
3. **데이터 불균형의 근본 문제**: 단순 loss weighting만으로는 불충분

11.3 실무 적용 시사점

- **일반 얼굴 분할**: 현재 모델로도 충분한 성능 (주요 부위 IoU > 0.85)
- **정밀 응용 (메이크업, 의료)**: 희소 클래스 개선 필수

- **실시간 처리**: 경량화 필요 (MobileNet 등)

본 실습은 전이학습의 효과와 클래스 불균형 문제의 도전 과제를 모두 경험할 수 있는 유의미한 과정이었다.